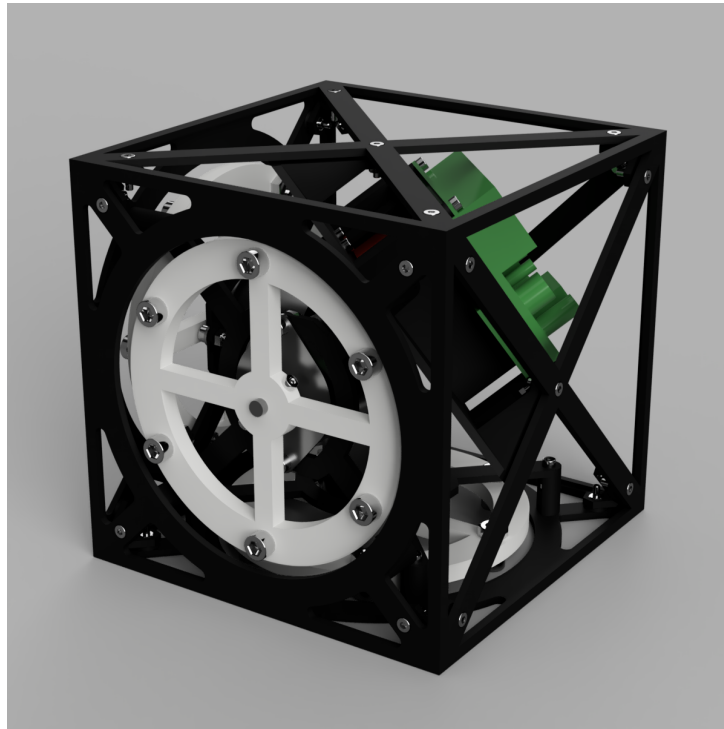


CubiloX Mk1

A Self-Balancing Reaction Wheel Cube



Technical Documentation

William Rosenberg

2026

This document is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

Abstract

The Cubilox Mk1 is a self-balancing cube that can stand on one of its edges or on one of its corners (vertices). Three reaction wheels, each driven by a brushless Nidec 24H motor, are mounted inside a 3D-printed frame. By accelerating and decelerating these wheels, the cube produces reaction torques that keep its centre of mass above the edge or the corner it is standing on. An ESP32 microcontroller estimates the orientation of the cube from an inertial measurement unit (IMU) and runs two controllers: a single-axis controller for the edge and a three-dimensional controller for the vertex, which separates the tilt motion from the rotation about the vertical axis.

The project was designed and built entirely by the author: the mechanical parts were designed in Fusion 360 and printed in PLA, the electronics were soldered on a prototype board, and the firmware was written from scratch. It was inspired by the Cubli of ETH Zurich [1], [2] and, more directly, by the self-balancing cube of remrc [3].

This document describes the working principle, the mechanical and electrical design, the firmware and its control laws, how to build, flash, calibrate and tune the cube, the results that were achieved, and the lessons learned during development. The final version balances on its edge and, after a calibration by hand, on its vertex for several minutes.

Contents

Abstract	i
1 Introduction and Inspiration	1
1.1 Motivation	1
1.2 Scientific background: the Cubli	1
1.3 Goals and scope of the Cubilox Mk1	1
1.4 Structure of this document	2
2 Working Principle and Physics	3
2.1 The cube as an inverted pendulum	3
2.2 Reaction wheels	3
2.3 Balancing on an edge (one axis)	3
2.4 Why wheel speed feedback is needed	4
2.5 Balancing on a vertex (three axes)	4
2.6 Angular momentum about the vertical axis	5
2.7 Sensing the orientation	5
3 Mechanical Design	6
3.1 Overview	6
3.2 Design for 3D printing	6
3.3 Reaction wheels	7
3.4 Sensor mount and sensor position	7
3.5 Battery compartment	9
4 Electronics	10
4.1 Overview	10
4.2 Power supply	12
4.2.1 Buffer capacitors	12
4.3 Battery voltage measurement	12
4.4 Buzzer driver	13
4.5 Pin assignment	13
4.6 Motor connection	13
4.7 Construction of the board	14
4.8 Safety note on the battery connection	15
5 Firmware and Control	16
5.1 Source files	16
5.2 Start-up	16
5.3 Main loop and timing	17
5.4 State machine	17
5.5 Sensor processing	17
5.5.1 Edge angle	17

5.5.2	Vertex tilt vector	17
5.6	Edge controller	18
5.6.1	Automatic trim	18
5.7	Vertex controller	18
5.8	Calibration and persistent storage	19
5.8.1	Gyroscope	19
5.8.2	Edge and vertex poses	19
5.9	Communication and logging	19
5.10	Where to change what	20
6	Getting Started	21
6.1	Software requirements	21
6.2	Flashing the firmware	21
6.3	Connecting	21
6.4	First start	22
6.5	Beep signals	22
6.6	Commands	23
7	Tuning Guide	24
7.1	Effect of the gains	24
7.2	Recommended procedure	25
8	Assembly	26
8.1	Reaction wheels and motors	26
8.2	Frame	26
8.3	Electronics	26
8.4	Commissioning	27
9	Results	28
9.1	Edge balancing	28
9.2	Vertex balancing	28
9.3	Motor test	29
9.4	Development history and lessons learned	29
10	Limitations	31
11	Outlook: CubiloX Mk2	32
	References	33
A	Bill of Materials	34
B	Licenses and Third-Party Software	36
B.1	Licenses of this project	36
B.2	Third-party software	36

List of Figures

1.1	The Cubilox Mk1: CAD model and built prototype.	2
2.1	Tilt error at the vertex: $\mathbf{r}$ is the gravity direction in the calibrated balance position, $\mathbf{n}$ the current one, both measured in the body frame. Their cross product is the rotation vector of the tilt.	5
3.1	Examples of geometry optimised for FDM printing.	7
3.2	Reaction wheel with the M6 screws and nuts on its rim and the two clamping screws at the hub.	8
3.3	Sensor mount and position of the IMU inside the cube.	8
4.1	Complete schematic of the Cubilox Mk1 (KiCad). The transistor Q1 is a 2N2222 in the built cube.	11
4.2	The prototype board: ESP32 board (right), buck converter (left), buzzer, voltage divider, buffer capacitors and the pin headers for the motors and the IMU. . . .	14
4.3	Motor cable extended with jumper wires, insulated with heat shrink tubing. . . .	15
9.1	Balancing on the edge.	28
9.2	Balancing on the vertex.	29

List of Tables

3.1	3D-printed parts of the Cubilox Mk1.	6
3.2	Key mechanical parameters.	6
4.1	Power distribution.	12
4.2	ESP32 pin assignment.	13
4.3	Nidec 24H motor connector (identical for all three motors).	14
5.1	Source files of the firmware.	16
5.2	Data stored in the flash memory (ESP32 Preferences).	20
5.3	Debug output (<code>debug 1</code>).	20
6.1	Beep signals of the buzzer.	22
6.2	Commands (USB or Bluetooth, one command per line).	23
7.1	Tunable gains and their default values.	24
9.1	Development history of the vertex balancing.	29
A.1	Bill of materials.	34
B.1	Licenses of the parts of this project (copyright 2026 William Rosenberg).	36
B.2	Third-party software used by the firmware.	36

Chapter 1

Introduction and Inspiration

1.1 Motivation

The Cubilox Mk1 is the author's first large electronics project. Before it, only small Arduino projects had been built. The idea for the cube goes back several years, to a video of the *Cubli*, a cube developed at the Institute for Dynamic Systems and Control of ETH Zurich that can jump up and balance on its corner [1], [2]. The goal was to build a similar cube completely independently, and to learn the required electronics, control engineering and mechanical design along the way.

While researching what such a cube needs, the author came across the self-balancing cube of remrc [3]. This project had a strong influence on the Cubilox Mk1: it uses the same type of motor (Nidec 24H) and a similar general layout. Nevertheless, every part of the Cubilox Mk1 was created by the author: all mechanical parts were designed from scratch, the circuit was designed and soldered independently, and the firmware was written without using code from other cube projects.

Because it is a first project of this size, not every solution is best practice, and some decisions would be made differently today. These points are discussed openly in Chapters 10 and 11. The cube works, and the knowledge gained here is the basis for future projects.

1.2 Scientific background: the Cubli

The Cubli is a 15 cm cube with reaction wheels mounted on three of its faces. By applying controlled torques to the wheels, it balances on an edge or a corner; by spinning the wheels up and braking them abruptly, it can even jump up from a resting position [1]. In the second publication, the multi-body dynamics of the Cubli are derived, its parameters are identified, and corner balancing with a linear state feedback controller is demonstrated experimentally [2]. The Cubli can therefore be seen as a three-dimensional version of the classical inverted pendulum, actuated purely by internal reaction wheels.

1.3 Goals and scope of the Cubilox Mk1

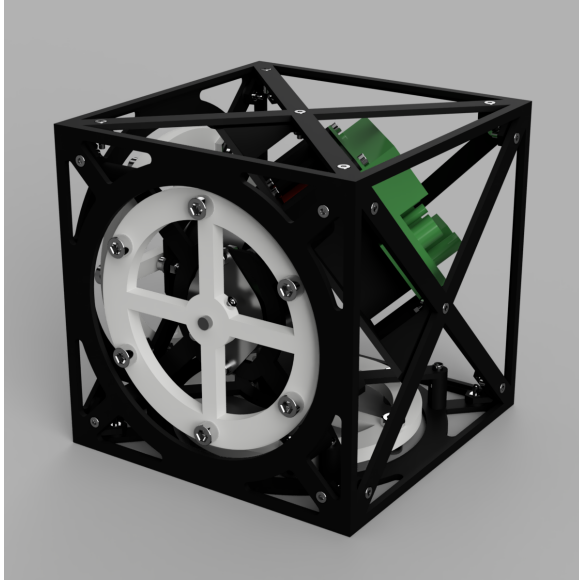
The goals of the project were:

- a cube that balances on an edge using one reaction wheel,
- a cube that balances on a vertex using all three reaction wheels,
- a design that can be printed on a standard consumer 3D printer and built from easily available parts,

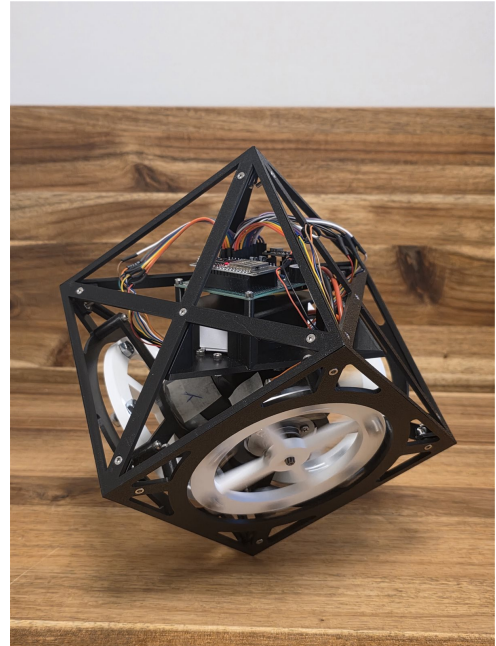
- self-written firmware that can be tuned and calibrated wirelessly over Bluetooth.

Jumping up from a resting position, as the Cubli does, was not a goal of the Mk1 (see Chapter 11).

Figures 1.1a and 1.1b show the CAD model and the finished cube balancing on its vertex.



(a) Rendering of the CAD model.



(b) The finished cube balancing on its vertex.

Figure 1.1: The Cubilox Mk1: CAD model and built prototype.

1.4 Structure of this document

Chapter 2 explains the physics of reaction wheel balancing. Chapters 3 and 4 describe the mechanical design and the electronics. Chapter 5 explains the firmware and the control laws in detail, followed by a getting started guide (Chapter 6) and a tuning guide (Chapter 7). Chapter 8 gives assembly instructions. Chapter 9 presents the results and the development history, Chapter 10 the limitations, and Chapter 11 the ideas for a Mk2. The bill of materials and the third-party software are listed in the appendix.

Chapter 2

Working Principle and Physics

This chapter explains why a cube can balance on an edge or a corner by means of internal reaction wheels. The equations are simplified models that explain the behaviour and the structure of the controllers in Chapter 5; the gains of the Cubilox Mk1 were tuned experimentally and not derived from an identified model.

2.1 The cube as an inverted pendulum

A cube resting on one of its edges or corners is an *inverted pendulum*: its centre of mass lies above the pivot, and any small deviation from the upright position produces a gravitational torque that increases the deviation. For a cube with edge length a and its centre of mass in the geometric centre, the distance between pivot and centre of mass is

$$l_{\text{edge}} = \frac{a}{\sqrt{2}} \quad \text{and} \quad l_{\text{vertex}} = \frac{\sqrt{3}}{2} a. \quad (2.1)$$

For the Cubilox Mk1 with $a = 157$ mm, and assuming the centre of mass in the geometric centre, this gives $l_{\text{edge}} \approx 111$ mm and $l_{\text{vertex}} \approx 136$ mm. In the balanced edge position the faces adjacent to the edge are inclined by 45° ; in the balanced vertex position the space diagonal through the pivot corner is vertical, so each face normal makes an angle of $\arccos(1/\sqrt{3}) \approx 54.7^\circ$ with the vertical.

2.2 Reaction wheels

A reaction wheel is a flywheel driven by a motor that is fixed to the body. When the motor applies a torque τ to the wheel, the wheel applies the opposite torque $-\tau$ to the body (Newton's third law). Accelerating a wheel therefore rotates the cube in the opposite direction, without any contact to the ground other than the pivot. The torque is only available while the wheel speed changes: a wheel spinning at constant speed produces no torque. Since the speed of a real motor is limited, a wheel cannot be accelerated in the same direction forever. This leads to the need for wheel speed feedback, explained in Section 2.4.

The reaction torque is proportional to the moment of inertia of the wheel and its angular acceleration. A large moment of inertia with little mass is achieved by placing mass far from the axis; this is why the wheels of the Cubilox Mk1 carry screws and nuts on their rim (Section 3.3).

2.3 Balancing on an edge (one axis)

On an edge, the cube can only rotate about the edge, and one reaction wheel whose axis is parallel to that edge is sufficient. Let θ be the tilt angle measured from the balance point, Θ

the moment of inertia of the cube about the edge, m its mass, g the gravitational acceleration, Θ_w the moment of inertia of the wheel and ω_w the speed of the wheel relative to the body. A simplified planar model is

$$\Theta \ddot{\theta} = mg l_{\text{edge}} \sin \theta - \tau, \quad (2.2)$$

$$\Theta_w (\dot{\omega}_w + \ddot{\theta}) = \tau, \quad (2.3)$$

where τ is the motor torque acting on the wheel. Linearised around $\theta = 0$ ($\sin \theta \approx \theta$), the uncontrolled system has a real positive eigenvalue $\lambda = \sqrt{mg l_{\text{edge}} / \Theta}$: without control, a small deviation grows exponentially and the cube falls.

A linear state feedback of the tilt angle, the tilt rate and the wheel speed stabilises the system:

$$u = K_1 \theta + K_2 \dot{\theta} + K_3 \omega_w, \quad (2.4)$$

where u is the motor command. In the firmware, these three terms are the gains **ek1**, **ek2** and **ek3** (Section 5.6):

- the **angle term** $K_1 \theta$ pushes the cube back towards the balance point,
- the **rate term** $K_2 \dot{\theta}$ damps the motion and prevents overshooting,
- the **wheel speed term** $K_3 \omega_w$ keeps the wheel speed bounded.

2.4 Why wheel speed feedback is needed

If the balance point assumed by the controller differs slightly from the real one (for example because the centre of mass is not exactly where it was expected), the cube needs a permanent torque to stay upright. A permanent torque means a permanently accelerating wheel, which sooner or later reaches its maximum speed; from then on, no torque is available and the cube falls. This is called *saturation*.

The wheel speed term solves this: when the wheel spins in one direction, the controller lets the cube lean slightly to the other side, so that gravity provides the torque instead of the wheel, and the wheel slows down again. In steady state $\dot{\theta} = 0$ and the motor command averages to zero, so Eq. (2.4) gives

$$K_1 \theta + K_3 \omega_w \approx 0. \quad (2.5)$$

This relation is also the basis of the automatic trim (Section 5.6.1): if the wheel keeps turning in one direction, the setpoint of the angle is shifted until the wheel rests on average.

2.5 Balancing on a vertex (three axes)

On a vertex, the cube can tilt in any direction, and three reaction wheels with mutually orthogonal axes are used. The orientation error is now a vector. Let $\mathbf{r}$ be the unit vector of the gravity direction in the balanced position and $\mathbf{n}$ the current gravity direction, both expressed in the body frame of the cube (Fig. 2.1). For small angles, the cross product

$$\mathbf{e} = \mathbf{n} \times \mathbf{r} \quad (2.6)$$

is the rotation vector (axis times angle, in radians) that describes the tilt of the cube. Its magnitude is the tilt angle and its direction is the axis about which the cube must be rotated back.

Rotations about $\mathbf{r}$, the vertical axis (*yaw*), are special: gravity produces no torque about this axis, so a yaw rotation does not affect the balance at all. The vertex controller therefore splits every vector quantity $\mathbf{v}$ into a tilt part and a yaw part:

$$\mathbf{v}_{\text{yaw}} = (\mathbf{v} \cdot \mathbf{r}) \mathbf{r}, \quad \mathbf{v}_{\text{tilt}} = \mathbf{v} - \mathbf{v}_{\text{yaw}}. \quad (2.7)$$

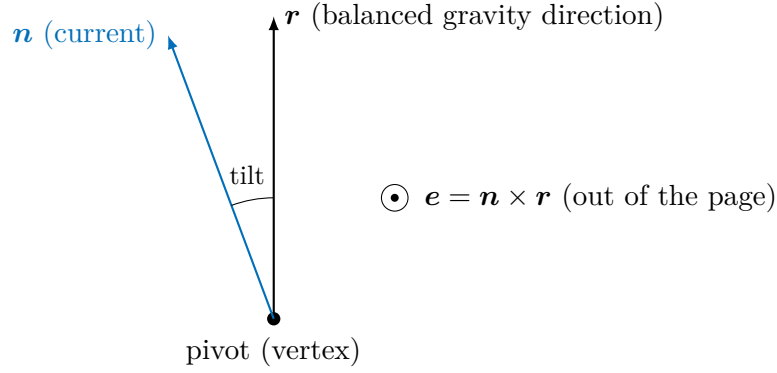


Figure 2.1: Tilt error at the vertex: $\mathbf{r}$ is the gravity direction in the calibrated balance position, $\mathbf{n}$ the current one, both measured in the body frame. Their cross product is the rotation vector of the tilt.

2.6 Angular momentum about the vertical axis

The wheels store angular momentum $\mathbf{h}$. Its tilt part is controlled exactly like the wheel speed on the edge: gravity can remove it if the cube leans slightly. Its yaw part, however, cannot be removed by gravity. If the controller fed back the yaw momentum in the same way as the tilt momentum, it would form a positive feedback loop: more yaw momentum would cause a larger command along the yaw axis, which would accelerate the wheels further. This effect was observed on the real cube (Section 9.4): with a negative yaw gain the yaw momentum grew exponentially with a time constant of about 2.4s until the motors saturated, and the cube was visibly rotating about its vertical axis.

The vertex controller therefore uses separate gains for the yaw part: a damping term on the yaw rate and a term that slowly reduces the yaw momentum of the wheels. The reaction is transferred to the ground through the friction at the pivot.

2.7 Sensing the orientation

The orientation is measured with an IMU that contains a three-axis accelerometer and a three-axis gyroscope:

- The **accelerometer** measures the gravity direction, which directly gives the tilt. However, it also measures every linear acceleration of the sensor. Each time the wheels accelerate, the cube jerks slightly, and the measured direction is disturbed.
- The **gyroscope** measures the angular rate. It is fast and not disturbed by linear accelerations, but integrating it to get an angle accumulates a slowly growing error (drift).

A *complementary filter* combines both: the gyroscope is integrated for fast changes, and the accelerometer slowly corrects the drift. With a filter constant α close to 1,

$$\hat{\theta}_k = \alpha (\hat{\theta}_{k-1} + \omega \Delta t) + (1 - \alpha) \theta_{\text{acc}}. \quad (2.8)$$

The firmware uses this filter on the edge and a three-dimensional version of it on the vertex (Section 5.5).

Chapter 3

Mechanical Design

3.1 Overview

All mechanical parts of the Cubilox Mk1 were designed by the author in Fusion 360 and printed in standard PLA on a Bambu Lab A1 printer, using the default settings of the Bambu Studio slicer. The frame of the cube consists of three motor walls carrying the motors and reaction wheels, three side walls, and eight corner mounts that join the walls with M3 screws and nuts. The electronics sit on a base plate inside the cube. Table 3.1 lists all printed parts and Table 3.2 the key parameters.

Table 3.1: 3D-printed parts of the Cubilox Mk1.

Part	Quantity	Function
motor_wall	3	frame face carrying one motor and its reaction wheel
side_wall	3	frame face without motor
corner_mount	8	joins the walls at the corners (M3 nuts press-fit)
motor_mount	6	holds a motor (fixed with M4 screws)
flywheel	3	reaction wheel
sensor_mount	1	holds the IMU near the centre of the cube
electronic_base_plate	1	carries the electronics
intermediate_plate	1	encloses the battery and carries the circuit board

Table 3.2: Key mechanical parameters.

Parameter	Value
Total mass of the cube	1036 g
Mass of one reaction wheel incl. screws and nuts	76.4 g
Mass of the battery	approx. 80 g
PLA used for all printed parts	283 g
Total print time	approx. 14 h
Edge length of the cube	157 mm

3.2 Design for 3D printing

All parts were designed to be printable without or with very few supports. Overhangs are kept at printable angles, and horizontal holes and nut pockets use teardrop or pointed shapes, so

that the upper side of the opening is built as a self-supporting overhang instead of a flat bridge. Figure 3.1 shows two examples.



(a) Nut pocket with a pointed roof, printable without supports.



(b) Detail of a printed frame part with a screw hole and cut-out.

Figure 3.1: Examples of geometry optimised for FDM printing.

3.3 Reaction wheels

Each reaction wheel (Fig. 3.2) is a printed ring with four spokes. Six M6 screws, each with two M6 nuts, are mounted on the rim. They add mass at the largest possible radius and thereby increase the moment of inertia of the wheel without making the wheel much heavier. One complete wheel including all screws and nuts weighs 76.4 g. The hub is clamped onto the motor shaft with two screws.

The wheel must sit exactly perpendicular on the motor shaft. A wheel that wobbles causes vibrations, which disturb the IMU, and it can rub against the frame. During development, a wheel that rubbed against the frame because of a loose screw produced cracking noises and shocks that shortened the balancing time considerably (Section 9.4).

3.4 Sensor mount and sensor position

The IMU is mounted as close as possible to the geometric centre of the cube, with each of its three axes aligned with the axis of one motor. This is why the motors are called X, Y and Z: motor X rotates the cube about the sensor's X axis, and so on. Placing the sensor near the centre keeps the accelerations it measures during rotations small.

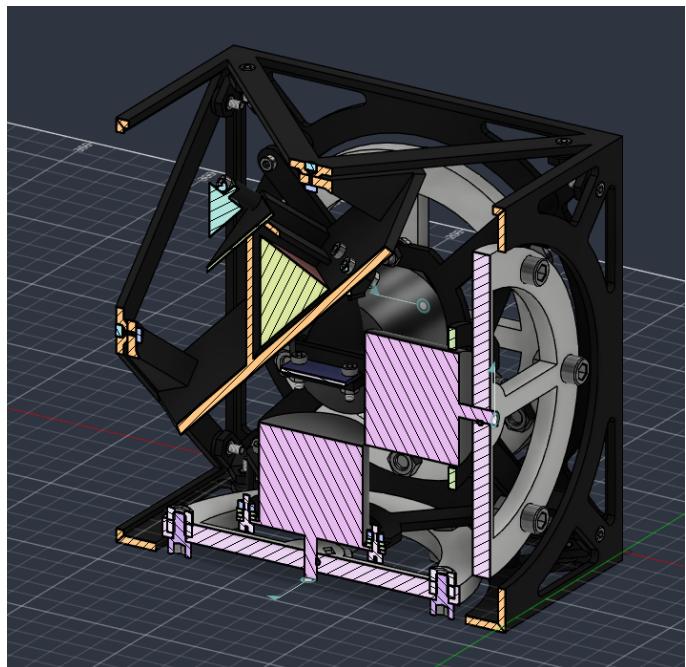
The sensor mount (Fig. 3.3) was created with the generative design function of Fusion 360. Despite its organic shape, it can be printed with remarkably little support material.



Figure 3.2: Reaction wheel with the M6 screws and nuts on its rim and the two clamping screws at the hub.



(a) Printed sensor mount with the IMU board.



(b) Section view of the CAD model showing the sensor position.

Figure 3.3: Sensor mount and position of the IMU inside the cube.

3.5 Battery compartment

The battery sits in a compartment that is closed by the intermediate plate. The circuit board is screwed on top of this plate with some clearance, so that the cables underneath the board are not crushed. The compartment is padded with cardboard and folded paper. The padding fixes the battery in place and absorbs some of the shock when the cube falls over. The accelerations during a normal fall are usually not high enough to damage the battery, but securing a LiPo battery against shocks is good practice.

Chapter 4

Electronics

4.1 Overview

All electronic components are soldered onto a 60 mm × 80 mm prototype board (Fig. 4.2). The main components are:

- an **ESP32 development board** (HW-394, ESP32-32D module, USB-C) as the microcontroller,
- an **IMU breakout board** labelled “MPU-9250/6500/9255” (MPU-6500 family, three-axis accelerometer and gyroscope), connected via I²C and read with the MPU6050_light library [4], whose register accesses are compatible with this sensor,
- three **Nidec 24H brushless motors** with integrated driver and encoder; the **8-pin version** is required (the author first bought the 12-pin version, which does not fit this design),
- a **3S LiPo battery** (11.1 V nominal, 12.6 V fully charged, 1000 mAh) with an XT30 connector,
- a **mini DC-DC buck converter** (input 5 V to 30 V, output 5 V),
- an **active 5 V buzzer** switched by a 2N2222 transistor,
- a **voltage divider** for measuring the battery voltage, and buffer capacitors.

The board carries three 8-pin headers for the motor cables and a 4-pin header for the IMU. The complete schematic is shown in Fig. 4.1; all connections are also listed in the file `hardware/electronics/wiring.md` of the repository.

The schematic was the author’s first project in KiCad and is therefore not as clearly arranged as it could be. Two details differ from the built cube: the transistor symbol is labelled 2N2219, while a 2N2222 is used, and the motors are numbered 1–3 instead of X, Y, Z (Nidec24H1 = motor X, Nidec24H2 = motor Z, Nidec24H3 = motor Y).

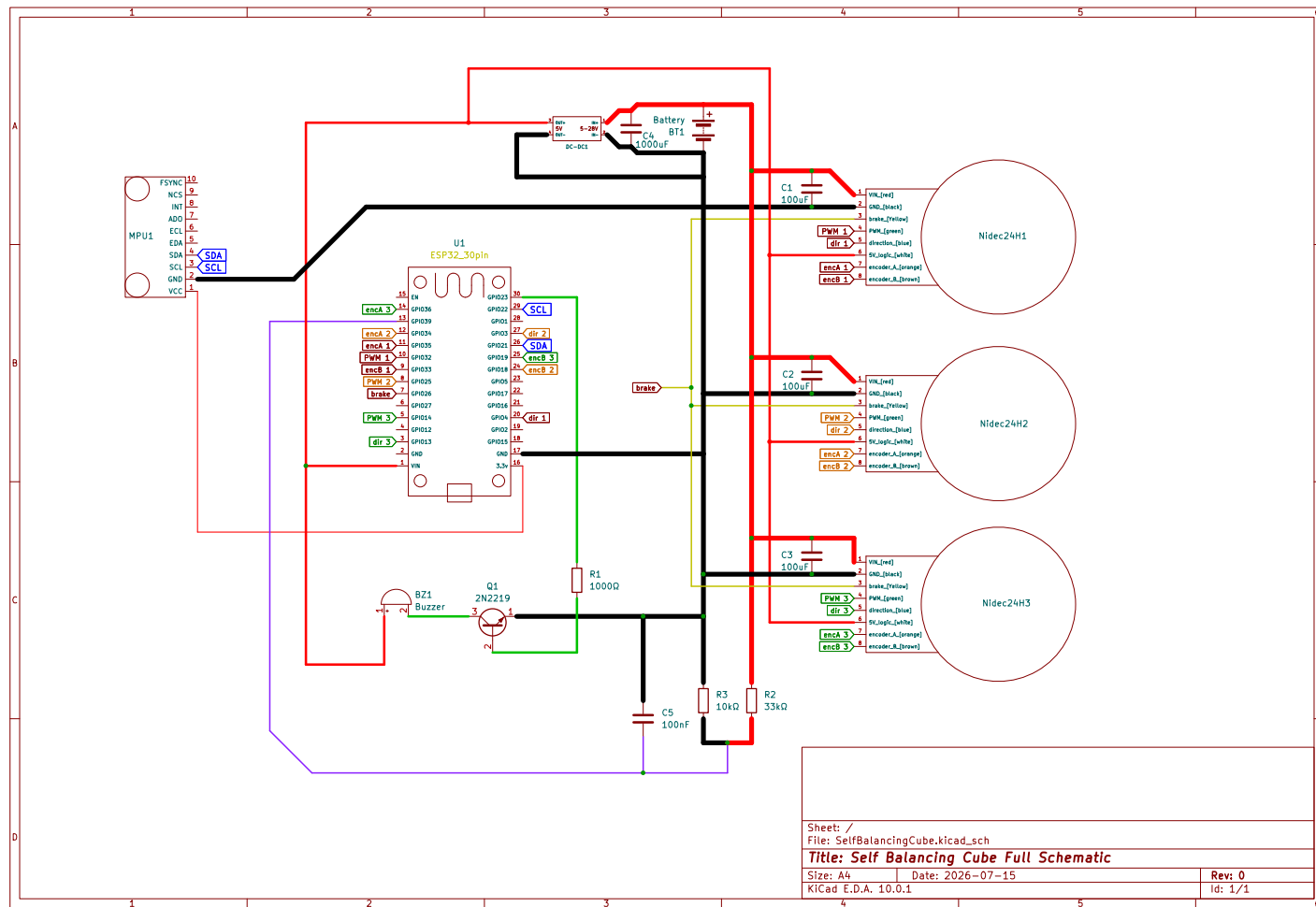


Figure 4.1: Complete schematic of the CubiloX Mk1 (KiCad). The transistor Q1 is a 2N2222 in the built cube.

4.2 Power supply

The battery feeds a common 12 V rail, which supplies the three motors directly (red wire, VIN) and the buck converter. The buck converter produces 5 V for the VIN pin of the ESP32 board and for the logic supply of the motors (white wire). The ESP32 board generates the 3.3 V for the IMU with its on-board regulator. All grounds are connected to one common GND rail.

Table 4.1: Power distribution.

Rail	Consumers
Battery (11.1 V to 12.6 V)	motor power (VIN) of all three motors, buck converter input, voltage divider
5 V (buck converter)	ESP32 VIN, motor logic supply, buzzer
3.3 V (ESP32 board)	IMU

4.2.1 Buffer capacitors

The motors draw short, high current peaks whenever they accelerate or reverse, which happens continuously while balancing. Because of the resistance and inductance of the battery and the wires, these peaks cause short voltage dips on the supply rails. Such dips can disturb or reset the ESP32 and the motor drivers. Three types of capacitors reduce this effect:

- **C4**, 1000 μF **electrolytic**, directly at the battery input: a large reservoir that supplies the current peaks of all motors together and smooths the main rail.
- **C1–C3**, 100 μF **electrolytic**, one at the supply of each motor: local reservoirs close to the consumers, so that the peak current of one motor does not have to flow through the whole wiring and disturb the others.
- **C5**, 100 nF **ceramic**, at the measuring point of the voltage divider: a small capacitor that filters noise on the ADC input (see Section 4.3).

4.3 Battery voltage measurement

The ESP32's analogue-to-digital converter (ADC) can only measure voltages up to about 3.3 V, while the battery reaches 12.6 V. A voltage divider scales the battery voltage down:

$$V_{\text{pin}} = V_{\text{bat}} \frac{R_3}{R_2 + R_3} = V_{\text{bat}} \frac{10 \text{ k}\Omega}{33 \text{ k}\Omega + 10 \text{ k}\Omega} \approx 0.233 V_{\text{bat}}. \quad (4.1)$$

For a full battery, $V_{\text{pin}} = 12.6 \text{ V} \times 0.233 \approx 2.93 \text{ V}$, safely below the limit. The firmware reverses this calculation by multiplying the measured pin voltage by $(R_2 + R_3)/R_3 = 4.3$ (Listing 4.1). The upper resistor of 33 k Ω is built from three resistors in series (20 k Ω + 10 k Ω + 3.3 k Ω), because no single 33 k Ω resistor was at hand.

The high resistance values keep the current through the divider small, $I = 12.6 \text{ V}/43 \text{ k}\Omega \approx 0.29 \text{ mA}$, so the divider hardly drains the battery. Together with the source resistance of the divider, $R_2 \parallel R_3 \approx 7.7 \text{ k}\Omega$, the 100 nF capacitor C5 forms a low-pass filter with a cut-off frequency of

$$f_c = \frac{1}{2\pi (R_2 \parallel R_3) C_5} \approx 207 \text{ Hz}, \quad (4.2)$$

which suppresses switching noise from the motors and provides the charge that the ADC needs for a sample.

```

1 float readBatteryPinV() {
2     uint32_t sum = 0;
3     for (int i = 0; i < 16; i++) sum += analogReadMilliVolts(VBAT_PIN);
4     return sum / 16.0 / 1000.0;
5 }
6
7 void updateBattery() {
8     float v = readBatteryPinV() * VBAT_FACTOR; // VBAT_FACTOR = (33 + 10) / 10
9     batteryV = (batteryV <= 0) ? v : 0.8 * batteryV + 0.2 * v;
10 }

```

Listing 4.1: Battery measurement in `hardware.ino`: 16 samples are averaged and converted to the battery voltage.

The battery voltage is only measured while the cube is not balancing, because the voltage drops under load. Below 11.1 V (3.7 V per cell) the cube warns with three beeps, and below 10.5 V (3.5 V per cell) it refuses to start, to protect the battery. A fully charged battery noticeably increases the available motor power.

4.4 Buzzer driver

The active buzzer only needs to be switched on and off. Since an ESP32 pin can only supply a small current, the buzzer is switched by an NPN transistor (2N2222) on its low side: GPIO 23 drives the base through a 1 k Ω resistor, the emitter is connected to GND and the collector to the negative terminal of the buzzer, whose positive terminal is connected to 5 V. The base current is approximately $(3.3\text{ V} - 0.7\text{ V})/1\text{ k}\Omega \approx 2.6\text{ mA}$, enough to switch the transistor fully on.

4.5 Pin assignment

Table 4.2 lists the pin assignment as defined in `config.h`. Strapping pins (GPIO 0, 2, 5, 12, 15), the flash pins (GPIO 6–11) and the USB serial pins (GPIO 1, 3) were deliberately avoided. GPIO 34, 35, 36 and 39 are input-only pins without internal pull resistors; the encoders work on them without external pull-ups.

Table 4.2: ESP32 pin assignment.

Signal	Motor X	Motor Y	Motor Z	Other
PWM	32	14	25	
Direction	4	13	27	
Encoder A	35	36	34	
Encoder B	33	19	18	
Brake (shared)		26		
I ² C SDA / SCL (IMU)				21 / 22
Buzzer				23
Battery voltage (ADC)				39

4.6 Motor connection

The Nidec 24H contains its own driver: it only needs the supply voltages, a PWM signal for the speed, a direction signal and a brake signal, and it returns two encoder signals. Table 4.3 shows

the 8-pin connector. The colours of the wires can differ between suppliers; the pin order is what matters. Two details are important for the firmware:

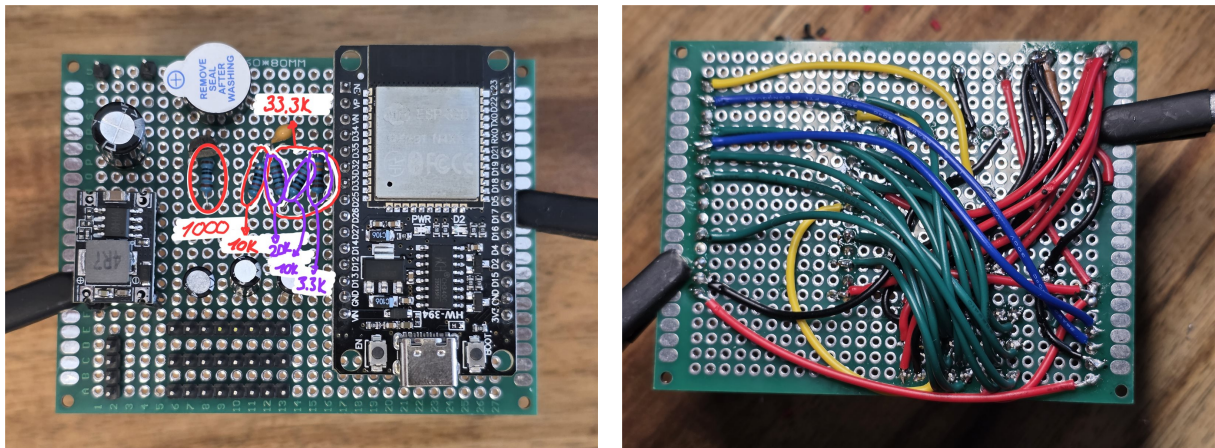
- the PWM input is inverted: a duty cycle of 255 (of 255) means “stop”, so the firmware writes $255 - |u|$,
- the brake input is active low: HIGH lets the motors run freely, LOW brakes them.

Table 4.3: Nidec 24H motor connector (identical for all three motors).

Pin	Colour	Function	Connected to
1	red	VIN (battery voltage)	12 V rail
2	black	GND	GND rail
3	yellow	brake	GPIO 26 (shared)
4	green	PWM	motor-specific GPIO
5	blue	direction	motor-specific GPIO
6	white	5 V logic supply	buck converter output
7	orange	encoder A	motor-specific GPIO
8	brown	encoder B	motor-specific GPIO

4.7 Construction of the board

Figure 4.2 shows both sides of the prototype board. The wiring on the bottom side is largely colour-coded: red for supply voltages, black for GND, blue for the two I²C lines of the IMU, yellow for the brake and the buzzer, and green for all encoder and PWM signals.



(a) Top side with the resistor values marked.

(b) Bottom side with the colour-coded wiring.

Figure 4.2: The prototype board: ESP32 board (right), buck converter (left), buzzer, voltage divider, buffer capacitors and the pin headers for the motors and the IMU.

The motor cables were extended with ordinary jumper wires with 2.54 mm female connectors, soldered to the motor wires and insulated with heat shrink tubing (Fig. 4.3). This takes some time but is cheap and works well. Alternatives are ready-made extension cables or crimping 8-pin cables with a matching connector kit.

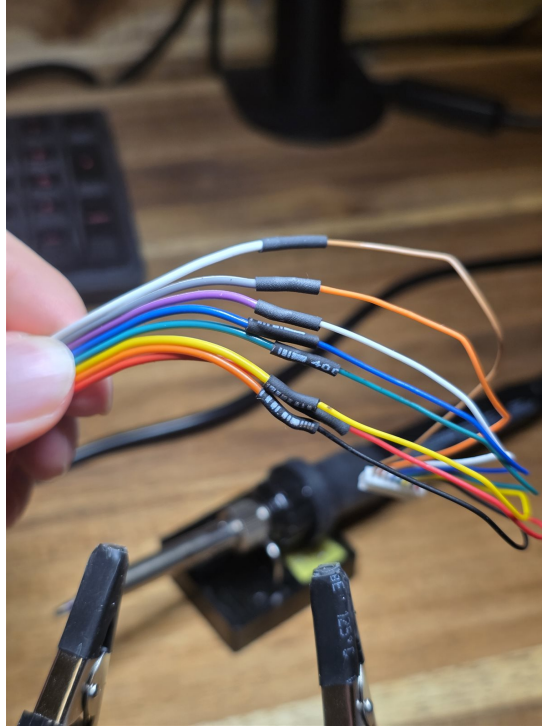


Figure 4.3: Motor cable extended with jumper wires, insulated with heat shrink tubing.

4.8 Safety note on the battery connection

The XT30 battery connector is soldered to the board via pin headers. These contacts are not insulated. They were useful during testing for measuring the battery voltage with a multimeter, but they are dangerous: touching them or accidentally bridging them causes a short circuit of the LiPo battery. **Do not copy this detail**; insulate all battery contacts.

Chapter 5

Firmware and Control

The firmware is written in C++ for the Arduino core for the ESP32 [5] and uses the MPU6050_light library [4] to read the IMU. This chapter explains its structure and the control laws. The firmware is located in the folder `firmware/cubilox_mk1/`.

5.1 Source files

Table 5.1: Source files of the firmware.

File	Content
<code>cubilox_mk1.ino</code>	<code>setup()</code> and <code>loop()</code> : start-up, sensor filters, state machine, edge and vertex controllers
<code>config.h</code>	pin definitions, controller gains, limits and timing constants, shared state variables
<code>calibration.ino</code>	gyroscope calibration at start-up, edge and vertex calibration, storage in flash, pose detection
<code>commands.ino</code>	command interface over USB and Bluetooth, storage of the gains
<code>hardware.ino</code>	log output, battery measurement, buzzer, motor control, encoder interrupts, motor test

The Arduino IDE combines all `.ino` files of the sketch folder into one program; global variables shared between the files are therefore declared in `config.h`.

5.2 Start-up

`setup()` performs the following steps:

1. start the USB serial port (115 200 baud) and Bluetooth with the device name `Cubilox_Mk1`,
2. load the gains and the calibration from flash,
3. configure the pins, attach the PWM outputs (20 kHz, 8 bit) and the encoder interrupts, stop all motors and engage the brake,
4. initialise the IMU (gyroscope range $\pm 500^\circ/\text{s}$, accelerometer range $\pm 2\text{g}$, the defaults of the library),
5. calibrate the gyroscope (the cube must lie still, see Section 5.8),
6. measure the battery voltage and report the calibration status.

After that, the cube waits until it is placed on a calibrated edge or vertex and starts balancing automatically.

5.3 Main loop and timing

`loop()` runs continuously. In every pass it processes incoming commands, reads the IMU and updates the sensor filters. The controllers themselves run whenever at least 5 ms have passed since the last control step, i.e. at up to 200 Hz. The debug output reports the longest gap between two control steps (`loop=`); on the real cube it is 5 ms to 6 ms.

The wheel speeds are computed from the encoder counts in a window of 20 ms and scaled by 5, so that their unit is “encoder counts per 100 ms”. All wheel speeds and wheel momenta in this chapter use this unit.

5.4 State machine

The firmware has three states:

WAITING The motors are off and braked. The firmware compares the measured acceleration vector with the calibrated edge and vertex poses. If it is closer than 0.25 g to one of them, the cube is close enough to the balance point (edge: $|\theta| < 1^\circ$, vertex: tilt $< 1.5^\circ$), it is at rest ($||\mathbf{a}| - 1\text{ g}| < 0.3\text{ g}$), the battery is not empty and these conditions hold for 400 ms, the brake is released and the corresponding balancing state is entered.

RUNNING_EDGE The edge controller drives motor X. The motor command is ramped up over 800 ms. Balancing stops if the angle exceeds 15° or the cube is moved ($||\mathbf{a}| - 1\text{ g}| > 0.3\text{ g}$).

RUNNING_VERTEX The vertex controller drives all three motors. The command is ramped up over 150 ms. Balancing stops if the tilt exceeds 25° , if a shock ($||\mathbf{a}| - 1\text{ g}| > 1.2\text{ g}$) lasts longer than 150 ms, or if at least one motor command stays above the maximum of 255 for more than 200 ms (saturation).

When balancing stops, all motors are switched off and braked, the run time and the cause are reported, and the state returns to WAITING.

5.5 Sensor processing

5.5.1 Edge angle

On the edge, only the rotation about the sensor’s X axis matters. The firmware uses the X tilt angle φ_x computed by the MPU6050_light library, subtracts the calibrated offset φ_0 and combines it with the gyroscope rate ω_x in a complementary filter:

$$\theta_k = 0.98 (\theta_{k-1} + \omega_x \Delta t) + 0.02 (\varphi_x - \varphi_0). \quad (5.1)$$

Here Δt is the time since the last control step, not since the last filter step. The edge gains were tuned with this behaviour, so it was deliberately left unchanged.

5.5.2 Vertex tilt vector

On the vertex, the tilt is a three-dimensional vector. The current gravity direction $\mathbf{n}$ is the normalised accelerometer vector, $\mathbf{r}$ the normalised calibrated vertex pose. The accelerometer-based tilt error (in degrees) is

$$\mathbf{e}_{\text{acc}} = \frac{180}{\pi} (\mathbf{n} \times \mathbf{r}). \quad (5.2)$$

The order of the cross product matters: $\mathbf{n} \times \mathbf{r}$ has the same sign as the integrated gyroscope rate (with the MPU6050_light conventions, a positive rotation about X increases the measured Y acceleration). The opposite order was the main bug during development (Section 9.4).

The tilt estimate $\mathbf{e}$ is obtained with a complementary filter with the time constant $\tau = 0.5$ s:

$$\mathbf{e}_k = a (\mathbf{e}_{k-1} + \boldsymbol{\omega} \Delta t) + (1 - a) \mathbf{e}_{\text{acc}}, \quad a = \frac{\tau}{\tau + \Delta t}, \quad (5.3)$$

where $\boldsymbol{\omega}$ is the gyroscope rate vector and Δt the measured time since the last filter step. The long time constant suppresses the disturbances of the accelerometer caused by the wheel accelerations. After every filter step, the component of $\mathbf{e}$ along $\mathbf{r}$ is removed, because a rotation about the vertical axis is not a tilt and is never corrected by the accelerometer.

5.6 Edge controller

The edge controller implements Eq. (2.4) for motor X:

$$u_X = \rho(t) (k_{e1} (\theta - \theta_{\text{trim}}) + k_{e2} \omega_x + k_{e3} n_X), \quad (5.4)$$

with the gains k_{e1} , k_{e2} , k_{e3} (`ek1`, `ek2`, `ek3`), the wheel speed n_X of motor X and the start-up ramp $\rho(t)$ that rises linearly from 0 to 1. The command u_X is limited to ± 255 .

```
1 float output = (ek1 * edgeAngle + ek2 * gyroRateX + ek3 * motorSpeed_X) * rampFactor;
2 motorControl_X((int)output);
```

Listing 5.1: Edge controller in `cubilox_mk1.ino`.

5.6.1 Automatic trim

If the calibrated balance point is slightly off, the wheel keeps turning in one direction (Section 2.4). According to Eq. (2.5), a positive wheel speed means that the angle setpoint is too large. The trim therefore moves the setpoint slowly against the wheel speed:

$$\theta_{\text{trim}} \leftarrow \theta_{\text{trim}} - k_{\text{trim}} n_X \Delta t, \quad (5.5)$$

limited to $\pm 3^\circ$, with the rate k_{trim} (`trim`, default 0.001). The trim is only active after the start-up ramp and is kept after a fall. On the real cube, the wheel oscillates around zero and the trim stays within about $\pm 0.1^\circ$.

5.7 Vertex controller

The vertex controller works in the coordinate frame of the cube. The motors Y and Z are mounted mirrored, which is described by the motor signs $s = (+1, -1, -1)$ and the encoder signs $w = (+1, +1, +1)$ in `config.h`. The physical wheel momentum vector is

$$\mathbf{h} = (s_X w_X n_X, s_Y w_Y n_Y, s_Z w_Z n_Z). \quad (5.6)$$

Both the gyroscope rate $\boldsymbol{\omega}$ and the wheel momentum $\mathbf{h}$ are split into a tilt part and a yaw part according to Eq. (2.7). The torque command vector is

$$\mathbf{t} = k_{v1} (\mathbf{e} - \mathbf{e}_{\text{trim}}) + k_{v2} \boldsymbol{\omega}_{\text{tilt}} + k_{v3} \mathbf{h}_{\text{tilt}} + (k_{yd} \omega_{\text{yaw}} - k_{yw} h_{\text{yaw}}) \mathbf{r}, \quad (5.7)$$

where $\omega_{\text{yaw}} = \boldsymbol{\omega} \cdot \mathbf{r}$ and $h_{\text{yaw}} = \mathbf{h} \cdot \mathbf{r}$. The gains are `vk1`, `vk2`, `vk3` for the tilt and `vyd`, `vyw` for the yaw axis. The three motor commands are

$$u_i = \rho(t) s_i t_i, \quad i \in \{X, Y, Z\}. \quad (5.8)$$

The first three terms of Eq. (5.7) are the three-dimensional version of the edge controller. The yaw term damps the rotation about the vertical axis and slowly removes the yaw momentum of the wheels; k_{yw} must be positive, otherwise it is a positive feedback (Section 2.6).

```

1 float yawTorque = vYawD * gYaw - vYawW * hYaw;
2 float tX = vK1 * (cfVertexAngleX - vTrimX) + vK2 * gTx + vK3 * hTx + yawTorque * rx;
3 float tY = vK1 * (cfVertexAngleY - vTrimY) + vK2 * gTy + vK3 * hTy + yawTorque * ry;
4 float tZ = vK1 * (cfVertexAngleZ - vTrimZ) + vK2 * gTz + vK3 * hTz + yawTorque * rz;
5
6 float outX = MOTOR_SIGN_X * tX * rampFactor;
7 float outY = MOTOR_SIGN_Y * tY * rampFactor;
8 float outZ = MOTOR_SIGN_Z * tZ * rampFactor;
```

Listing 5.2: Core of the vertex controller in `cubiloX_mk1.ino`.

The vertex has an automatic trim as well, which works like the edge trim but as a vector perpendicular to $\mathbf{r}$: $\mathbf{e}_{\text{trim}} \leftarrow \mathbf{e}_{\text{trim}} - k_{\text{vtrim}} \mathbf{h}_{\text{tilt}} \Delta t$, limited to 3° . Its rate `vtrim` is 0 (off) by default, because the cube balances best with a careful calibration by hand (Section 9.4).

5.8 Calibration and persistent storage

5.8.1 Gyroscope

The gyroscope has a small offset that changes with temperature. It is therefore measured at every start: the cube only has to lie still for about three seconds. The surface does not need to be level, because only the gyroscope is calibrated.

5.8.2 Edge and vertex poses

The balance points of the edge and the vertex are measured once with the commands `edge` and `vertex` and stored in flash. The user balances the cube by hand on the edge or vertex; after 7s (one short tick per second), the acceleration vector is averaged for 3s. The measurement is rejected if

- the standard deviation of one acceleration component exceeds 0.05 g (the cube was moved), or
- the measured direction is more than 12° away from an ideal pose: for the vertex a space diagonal $(\pm 1, \pm 1, \pm 1)/\sqrt{3}$, for the edge a diagonal in the Y–Z plane, i.e. an edge parallel to the axis of motor X.

Balancing by hand finds the *real* balance point, where the centre of mass is exactly above the pivot. A fixture that holds the cube geometrically straight does not, because the centre of mass is generally not exactly in the geometric centre. This difference was the key to stable vertex balancing (Section 9.4).

The accelerometer offsets are not recomputed at start-up. The offsets that were active during the calibration are stored together with the poses and loaded again at every start, so that the stored poses remain valid after every restart and power loss. (A constant accelerometer offset does not matter for balancing, because the poses are measured with the same offset.)

5.9 Communication and logging

Commands can be sent over the USB serial port or over Bluetooth Classic (serial port profile) with any serial terminal; they are listed in Section 6.6. All output goes to both interfaces.

Table 5.2: Data stored in the flash memory (ESP32 Preferences).

Namespace	Content	Deleted by
<code>cube</code>	controller gains (after <code>save</code>)	<code>reset</code>
<code>cuberef</code>	edge and vertex poses, accelerometer offsets	new calibration (overwrites)
<code>cubebat</code>	battery voltage factor (after <code>vbatcal</code>)	new <code>vbatcal</code>

During development, an analysis of the Bluetooth library showed that it sends every print call as a separate packet; with a connected client, its transmit queue holds 32 packets and a full queue can block the caller for up to one second. Since the diagnostic output consisted of many print calls, this could stall the control loop. The firmware therefore formats each log line completely and sends it as one packet (`logLine()`), and it always writes the motor commands *before* any log output.

In normal operation, the cube only reports the start and the end of a balancing run (with the run time and the cause) and a short status line every 10s. With `debug 1`, detailed lines are printed (Table 5.3).

Table 5.3: Debug output (`debug 1`).

Line	Fields
IDLE (every 500 ms)	detected pose, acceleration vector, edge angle, vertex tilt, battery voltage
EDGE (every 500 ms)	time <code>t</code> , <code>angle</code> , wheel speed <code>wheel</code> , <code>trim</code> , longest control gap <code>loop</code>
RUN (every 50 ms)	time <code>t</code> , tilt vector <code>e</code> , gyroscope <code>g</code> , wheel speeds <code>w</code> , motor commands <code>o</code> , yaw and tilt momentum, trim vector, <code>loop</code>
STOP	tilt, acceleration magnitude and number of short shocks at the end of a vertex run

5.10 Where to change what

Most settings are constants in `config.h`:

- **Pins** (`PWM_X`, `DIR_X`, ...): adapt them if the board is wired differently.
- **Default gains** (`eK1`–`eK3`, `vK1`–`vK3`, `vYawD`, `vYawW`, trim rates): they are used until gains are stored with `save`.
- **Motor and encoder signs** (`MOTOR_SIGN_*`, `WHEEL_SIGN_*`): must match the mounting direction of the motors; the command `motortest` recommends them.
- **Start and stop limits** (`EDGE_START_ANGLE_DEG`, `VERTEX_STOP_ANGLE_DEG`, ramp times, `SATURATION_STOP_MS`, ...).
- **Calibration timing** (`CALIB_SETTLE_MS` = 7 s, `CALIB_AVERAGE_MS` = 3 s) and its plausibility limits.
- **Battery** (`VBAT_FACTOR`, `BAT_WARN_V`, `BAT_MIN_V`).
- **Bluetooth name** (`BT_DEVICE_NAME`).

Chapter 6

Getting Started

This chapter describes how to flash the firmware, start the cube for the first time and calibrate it.

6.1 Software requirements

- Arduino IDE 2,
- Arduino core for the ESP32 [5], version 3.x (tested with 3.3.10; installed through the Boards Manager),
- library MPU6050_light [4] (tested with 1.2.1; installed through the Library Manager).

The libraries `BluetoothSerial`, `Preferences` and `Wire` are part of the ESP32 core.

6.2 Flashing the firmware

1. Open `firmware/cubiloX_mk1/cubiloX_mk1.ino` in the Arduino IDE. The folder name must match the name of the main file, so do not rename it.
2. Select the board “ESP32 Dev Module” and the default partition scheme (Bluetooth needs the default scheme; the firmware uses about 85 % of the program memory).
3. Connect the ESP32 via USB and upload the sketch.

With the Arduino command line tool, the equivalent command is

```
arduino-cli compile --fqbn esp32:esp32:esp32 firmware/cubiloX_mk1
arduino-cli upload --fqbn esp32:esp32:esp32 -p <port> firmware/cubiloX_mk1
```

6.3 Connecting

Commands can be sent in two ways:

- **USB:** serial monitor of the Arduino IDE, 115 200 baud, line ending “newline”. The USB cable disturbs the balance, so this is mainly useful on the desk.
- **Bluetooth:** pair the device `CubiloX_Mk1` and use a serial terminal app, for example “Serial Bluetooth Terminal” on Android. The ESP32 uses Bluetooth Classic (serial port profile), which iPhones do not support.

6.4 First start

1. Charge the battery fully and connect it.
2. Lay the cube on any side and do not touch it. After a short beep it calibrates the gyroscope (about 3 s), then two beeps follow, and a long beep signals that it is ready.
3. Connect via Bluetooth and send `status`. On the first start, it reports that the edge and the vertex are not calibrated.
4. Optional: run `motortest` while holding the cube loosely in your hand. All three encoder values should be similar (about 380 on the original cube) and the recommended signs should match `config.h`.
5. **Calibrate the edge:** send `edge`. Within 7 s, balance the cube by hand on an edge that is parallel to the axis of motor X, as still as possible. A long beep starts the measurement; hold the cube still for 3 s. Two beeps confirm the calibration.
6. **Calibrate the vertex:** send `vertex` and do the same on the vertex. Find the point where the cube balances by itself with almost no force from your fingers.
7. Place the cube on the edge or vertex and hold it gently near the balance point. When it detects the pose, it starts by itself (two short beeps).

The calibration is stored permanently. It only has to be repeated if something changes the centre of mass, for example if the battery is mounted in a different position. If the vertex balance is restless or the cube drifts away in one direction, a new `vertex` calibration usually helps; several attempts may be needed to find the best balance point.

6.5 Beep signals

Table 6.1: Beep signals of the buzzer.

Signal	Meaning
1 short beep at power-on	IMU found, gyroscope calibration follows
2 beeps, then 1 long beep	gyroscope calibrated, cube ready
2 short beeps	balancing started
1 beep (200 ms)	balancing stopped
1 beep, then ticks every second	calibration: balance the cube now
1 long beep (150 ms) during calibration	measurement starts, hold still
2 beeps after calibration	calibration saved
1 very long beep (800 ms)	calibration rejected
3 short beeps (every 30 s)	battery low (below 11.1 V)
1 beep every 2 s when placed on a pose	battery empty (below 10.5 V), no start
1-second beep every 2 s after power-on	IMU not found

6.6 Commands

Table 6.2: Commands (USB or Bluetooth, one command per line).

Command	Function
<code>help</code> or <code>?</code>	list of commands
<code>status</code> or <code>p</code>	current gains, calibration status, battery voltage, debug mode
<code>edge</code>	calibrate the edge balance point (Section 5.8)
<code>vertex</code>	calibrate the vertex balance point
<code><gain> <value></code>	set a gain, e.g. <code>vk1 95</code> (see Chapter 7)
<code><gain></code>	show a gain
<code>save</code>	store all gains in flash
<code>reset</code>	delete the stored gains and use the defaults from <code>config.h</code>
<code>debug 0</code> / <code>debug 1</code>	detailed log output off / on
<code>motortest</code>	short pulse on each motor; shows gyroscope reaction, encoder counts and the recommended motor and encoder signs
<code>vbatcal <V></code>	optional: fine-tune the battery measurement to a multimeter reading, e.g. <code>vbatcal 12.6</code>

Commands that write to flash or move the motors (`edge`, `vertex`, `save`, `reset`, `motortest`, `vbatcal`) are only accepted while the cube is not balancing. Changed gains are lost at the next restart unless they are stored with `save`.

Chapter 7

Tuning Guide

The gains can be changed at runtime over Bluetooth (for example `vk2 9`) and stored with `save`. Table 7.1 lists all gains with their default values, which are the values the cube balances with. The motor command is a PWM value between -255 and $+255$; angles are in degrees, rates in degrees per second and wheel speeds in encoder counts per 100 ms.

Table 7.1: Tunable gains and their default values.

Command	Default	Meaning
<code>ek1</code>	70	edge: angle gain (PWM per degree)
<code>ek2</code>	8	edge: rate gain (PWM per degree/s)
<code>ek3</code>	0.35	edge: wheel speed gain
<code>trim</code>	0.001	edge: rate of the automatic trim (0 = off)
<code>vk1</code>	90	vertex: tilt angle gain
<code>vk2</code>	9.5	vertex: tilt rate gain
<code>vk3</code>	0.30	vertex: wheel momentum gain (tilt part)
<code>vyd</code>	9.5	vertex: damping of the rotation about the vertical axis
<code>vyw</code>	0.15	vertex: reduction of the wheel momentum about the vertical axis
<code>vtrim</code>	0	vertex: rate of the automatic trim (0 = off)

7.1 Effect of the gains

Angle gain (`ek1`, `vk1`) determines how strongly the cube is pushed back towards the balance point. Too low: the cube reacts sluggishly, leans far and falls slowly to one side. Too high: it overreacts, oscillates quickly and the motors reach their limit.

Rate gain (`ek2`, `vk2`) damps the motion. Too low: the cube overshoots and oscillates. Too high: the motors react to sensor noise and switch rapidly back and forth; this is audible as a buzzing and produces short shocks. On the vertex, `vk2 11` caused such shocks, `vk2 9.5` did not.

Wheel speed gain (`ek3`, `vk3`) prevents the wheels from running away. Too low: the wheels slowly spin up until the motors saturate and the cube falls after a few seconds. Too high: a slow oscillation of the whole cube. During development, the vertex rocked slowly with a period of about 1.3s at 70 / 8 / 0.35; raising the angle and rate gains and slightly lowering the wheel gain (90 / 11 / 0.30, later 90 / 9.5 / 0.30) stabilised it.

Yaw damping (`vyd`) damps the rotation of the cube about its vertical axis.

Yaw momentum gain (vyw) slowly reduces the momentum of the wheels about the vertical axis. It *must be positive*. With a negative value (-0.3), the yaw momentum grew exponentially until the motors saturated after 7–10 s (Sections 2.6 and 9.4).

Trim rates (trim, vtrim) determine how fast the automatic trim corrects the balance point. The trim should be much slower than the balancing motion; a trim that is too fast can interact with the controller. On the vertex, a careful calibration by hand worked better than the trim, which is why `vtrim` is 0 by default.

7.2 Recommended procedure

1. Start with a fully charged battery: the available motor power depends noticeably on the battery voltage.
2. Calibrate first, then tune. Many problems that look like bad gains are actually a wrong balance point (Section 9.4).
3. Change only **one** gain at a time and test it several times. Changing two things at once made it impossible to tell which change caused an effect.
4. Use `debug 1` and record the output. Check that every wheel speed (`w=`) reacts to its motor command (`o=`); a wheel that stays at zero indicates a hardware problem such as a loose connector.
5. The cause reported at the end of a run helps: “motors saturated” usually means that the wheel speeds ran away (wheel speed gain, calibration or yaw gain), “tilt angle too large” that the cube fell over without the motors being at their limit (angle or rate gain), and “sustained shock” that the cube was hit or vibrates strongly.
6. Store good values with `save`.

Chapter 8

Assembly

The 3D files are published separately on MakerWorld (Cubilox Mk1 on MakerWorld). All parts listed in Table 3.1 are printed in PLA; the author used a Bambu Lab A1 with the default settings of Bambu Studio. The complete bill of materials is given in Appendix A.

8.1 Reaction wheels and motors

1. Mount all M6 screws and nuts on the rims of the three flywheels first (Fig. 3.2).
2. Push each flywheel onto its motor shaft. The fit is very tight; on the original cube, the bore of the flywheel was widened slightly with drill bits until it fitted exactly.
3. Clamp the flywheel to the shaft with the two hub screws. Make sure that the wheel sits exactly perpendicular on the shaft and does not wobble. A wobbling wheel causes vibrations and can rub against the frame.
4. Screw the motor mounts to the motors with M4 screws and nuts.

8.2 Frame

1. Press the M3 nuts into the corner mounts. They should be a light press fit, so that they do not fall out during assembly.
2. First assemble the three motor sides completely, then join the three sides using the corner mounts. This order makes the assembly much easier.
3. If an M3 nut falls out, small tweezers help to put it back; with the fingers it is hard to reach.

8.3 Electronics

1. Solder the board according to the schematic (Fig. 4.1) and Tables 4.2 and 4.3. Check all connections with a multimeter before connecting the battery for the first time.
2. Extend the motor cables (Fig. 4.3) and connect them to the 8-pin headers, keeping the pin order.
3. Mount the IMU on the sensor mount so that its X, Y and Z axes are parallel to the axes of motors X, Y and Z (Fig. 3.3).
4. Pad the battery compartment (for example with cardboard and folded paper) so that the battery sits firmly and is protected against shocks.

5. Insulate all battery contacts (see the safety note in Chapter 4).

8.4 Commissioning

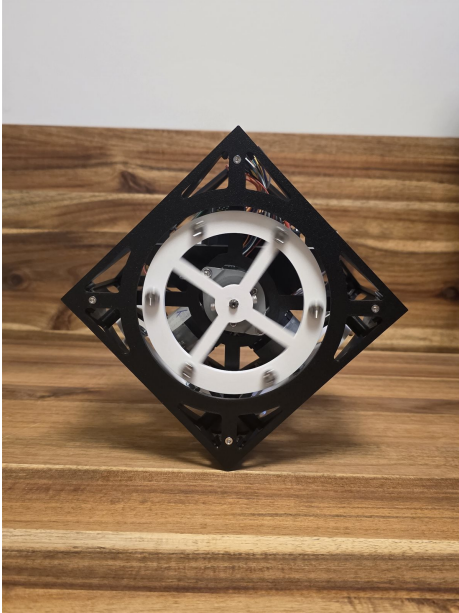
Flash the firmware, run `motortest` and check the motor and encoder signs, then calibrate the edge and the vertex as described in Chapter 6. If a motor of a rebuilt cube is mounted in the opposite direction, the signs in `config.h` must be adapted to the values recommended by `motortest`.

Chapter 9

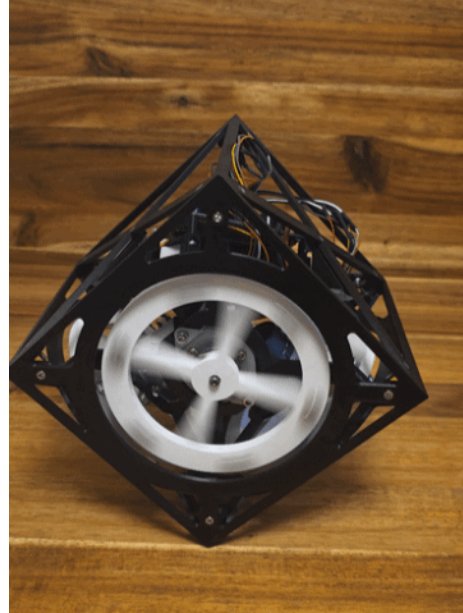
Results

9.1 Edge balancing

The edge balancing works reliably. With the automatic trim, the wheel oscillates around zero speed and the trim settles within about $\pm 0.1^\circ$. The cube also recovers from gentle pushes, as shown in the animation in the repository (Fig. 9.1).



(a) Front view of the cube balancing on its edge.

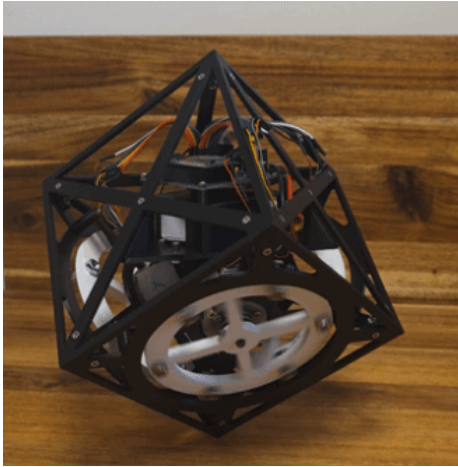


(b) Frame of the animation in which the cube is pushed while balancing on its edge.

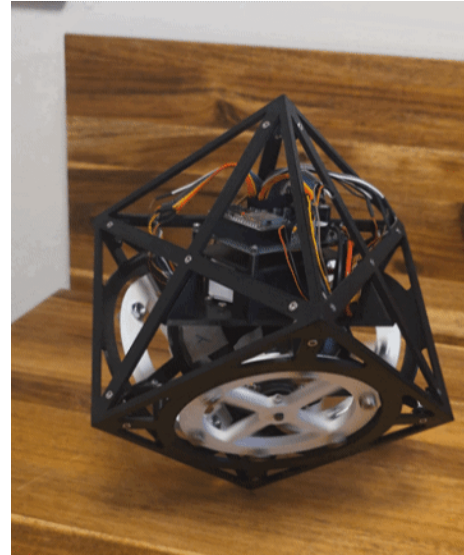
Figure 9.1: Balancing on the edge.

9.2 Vertex balancing

After the improvements described in Section 9.4, and in particular after calibrating the vertex by hand, the cube balances on its vertex for several minutes in a row. Only a small residual oscillation remains, which is attributed to the limited motor power and to a calibration that is not perfect. Figure 9.2 shows frames of the two animations in the repository.



(a) Frame of the balancing loop animation.



(b) Frame of the animation with a view around the cube.

Figure 9.2: Balancing on the vertex.

9.3 Motor test

The motor test (`motortest`, PWM 150 for 200 ms) gave encoder counts of 379, 377 and 380 for the motors X, Y and Z, confirming that the three motors deliver the same power, and the measured gyroscope reactions confirmed the motor signs $(+1, -1, -1)$ and the encoder signs $(+1, +1, +1)$.

9.4 Development history and lessons learned

The vertex balancing did not work at the first attempt. Table 9.1 summarises the most important problems, their causes and the resulting balancing times on the vertex. Several of these lessons apply to any balancing robot.

Table 9.1: Development history of the vertex balancing.

Problem	Cause and solution	Vertex time
Vertex filter drifted to the wrong sign after about 0.3 s	The tilt error was computed as $\mathbf{r} \times \mathbf{n}$ instead of $\mathbf{n} \times \mathbf{r}$, so gyroscope and accelerometer had opposite signs. Additionally, the filter used a wrong time step and included the yaw rotation. All three were fixed.	–
Cube slowly tipped away	The encoder signs of motors Y and Z were wrong, which cancelled half of the restoring action. Fixed with the motor test.	1–2 s
Slow rocking with a period of about 1.3 s	Gains changed from 70 / 8 / 0.35 to 90 / 11 / 0.30.	–
Motor Z stopped working in two runs	Loose contact (encoder showed zero despite a large command).	–

Problem	Cause and solution	Vertex time
Run stopped by the shock detection although the cube was upright	Fast reversals of the wheels produced short accelerations above 2 g; stopping and braking then threw the cube over. Now only a shock lasting longer than 150 ms stops the run; <code>vk2</code> reduced to 9.5.	–
Cracking noises	A wheel rubbed against the frame because of a loose screw.	7–8 s
Runs ended with the wheels saturated, cube visibly rotating about its vertical axis	The wheel momentum about the vertical axis was fed back like the tilt momentum (positive feedback). The yaw axis is now controlled separately (Section 2.6).	–
All run times suddenly much shorter (about 2 s)	Bluetooth output blocked the control loop. Every log line is now sent as one packet, after the motor commands. Afterwards, the measured control gap was a constant 6 ms.	7–10 s
Results varied strongly between sessions	The vertex pose had been calibrated in a fixture that held the cube geometrically straight, but the centre of mass was slightly off the geometric axis. Calibrating by hand, at the real balance point, solved the problem.	several minutes

The main lessons are:

- **Signs and conventions first.** Most early failures were sign errors. A motor test that measures the reaction of the cube to each motor saves a lot of time.
- **The balance point is defined by the centre of mass, not by the geometry.**
- **Look for positive feedback** when a quantity grows exponentially.
- **Logging must never block the control loop.**
- **Change only one thing at a time.** In one test series, two changes were made at once, and their effects could not be separated.
- **Mechanical problems look like control problems.** A rubbing wheel or a loose contact can ruin otherwise good gains.

Chapter 10

Limitations

The Cubilox Mk1 is the author's first large electronics project. It works, but several aspects are not best practice or limit its performance:

- **Motor power.** The available torque of the Nidec 24H motors is the main limit. Runs end when the motors saturate, a small residual oscillation remains on the vertex, and the cube cannot jump up by itself.
- **Empirical gains.** The gains were tuned by experiment. No model of the cube was identified, and no model-based controller (such as the linear state feedback of the Cubli [2]) was designed.
- **Calibration by hand.** The vertex balance point has to be found by balancing the cube by hand; the result depends on the user and may need several attempts. The automatic vertex trim did not replace this calibration.
- **Edge balancing on one axis.** Only edges parallel to the axis of motor X are supported.
- **Sensor disturbances.** The accelerometer is disturbed by every wheel acceleration; the vertex filter therefore relies on the gyroscope for fast motion and corrects the tilt only slowly.
- **Prototype board.** All electronics are hand-wired on a prototype board. The wiring takes space, adds weight and is error-prone (one loose contact was found during development).
- **Exposed battery contacts.** The XT30 connector is attached via uninsulated pin headers, which is a short-circuit risk.
- **Schematic.** The schematic was the author's first KiCad project and is not as clearly laid out as it could be; two labels differ from the built cube (Chapter 4).
- **Acoustic feedback only.** All status information is given by the buzzer, which became a lot of beeping during testing.

Chapter 11

Outlook: Cubilox Mk2

The experience from the Mk1 leads to the following plans for a Mk2:

- **A real printed circuit board** with proper connectors instead of the prototype board and soldered jumper wires. This saves space and weight and makes the wiring more reliable.
- **Stronger motors.** More torque would allow calmer balancing with more reserve and could make it possible for the cube to jump up from a resting position, like the Cubli [1].
- **An RGB status LED.** Start-up, calibration and state changes would be shown by light; the buzzer would only be used for important events such as errors or an empty battery.

Further possible improvements that follow from the limitations in Chapter 10:

- **Safe battery connection** with insulated contacts.
- **Model-based control.** With an identified model of the cube, the gains could be computed (for example with a linear-quadratic regulator) instead of being tuned only by experiment.

References

- [1] M. Gajamohan, M. Merz, I. Thommen, and R. D’Andrea, “The Cubli: A cube that can jump up and balance,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2012, pp. 3722–3727. DOI: 10.1109/IROS.2012.6385896
- [2] M. Gajamohan, M. Muehlebach, T. Widmer, and R. D’Andrea, “The Cubli: A reaction wheel based 3D inverted pendulum,” in *2013 European Control Conference (ECC)*, 2013, pp. 268–274.
- [3] remrc. “Self-Balancing-Cube.” GitHub repository, Accessed: Oct. 7, 2026. [Online]. Available: <https://github.com/remrc/Self-Balancing-Cube>
- [4] R. Fétick. “MPU6050_light: Arduino library for light and fast communication with the MPU6050.” Version 1.2.1, MIT License, Accessed: Oct. 7, 2026. [Online]. Available: https://github.com/rfetick/MPU6050_light
- [5] Espressif Systems. “Arduino core for the ESP32.” Version 3.3.10, Accessed: Oct. 7, 2026. [Online]. Available: <https://github.com/espressif/arduino-esp32>

Appendix A

Bill of Materials

The same list is available as `BOM.csv` in the root of the repository.

Table A.1: Bill of materials.

Item	Qty.	Specification / notes
Electronics		
ESP32 development board	1	HW-394, ESP32-32D module, 30 pins, USB-C
IMU breakout board	1	MPU-6500 (labelled MPU-9250/6500/9255)
Brushless motor Nidec 24H	3	8-pin connector version (integrated driver and encoder)
LiPo battery	1	3S, 11.1 V, 1000 mAh, approx. 63 mm $\times$ 30 mm $\times$ 24 mm, approx. 80 g
XT30 connector	1	battery connector
DC-DC buck converter (mini)	1	input 5 V to 30 V, output 5 V
Prototype PCB	1	60 mm $\times$ 80 mm
Electrolytic capacitor	1	1000 μ F, bulk capacitor at the battery input
Electrolytic capacitor	3	100 μ F, one per motor supply
Ceramic capacitor	1	100 nF, at the battery voltage divider
Resistor	1	33 k Ω (built as 20 k Ω + 10 k Ω + 3.3 k Ω in series)
Resistor	1	10 k Ω
Resistor	1	1 k Ω , base resistor of the buzzer transistor
NPN transistor 2N2222	1	low-side switch for the buzzer
Active buzzer	1	5 V
Pin headers	as req.	2.54 mm; 3 $\times$ 8-pin (motors), 1 $\times$ 4-pin (IMU), battery connector
Jumper wires	as req.	2.54 mm female, motor cable extensions
Heat shrink tubing	as req.	insulation of the soldered extensions
Hardware		
M6 nut	36	additional rim mass of the reaction wheels
M6 $\times$ 10 mm screw	18	additional rim mass of the reaction wheels
M4 nut	6	motor mounts
M4 $\times$ 10 mm screw	6	motor mounts
M3 nut	53	general assembly (frame and mounts)
M3 $\times$ 10 mm screw	53	general assembly (frame and mounts)
3D-printed parts (PLA)		
motor_mount	6	

Item	Qty.	Specification / notes
flywheel	3	
motor_wall	3	
side_wall	3	
corner_mount	8	
sensor_mount	1	
electronic_base_plate	1	
intermediate_plate	1	encloses the battery, carries the circuit board

Appendix B

Licenses and Third-Party Software

B.1 Licenses of this project

Table B.1: Licenses of the parts of this project (copyright 2026 William Rosenberg).

Part	Folder	License
Firmware and source code	<code>firmware/</code>	MIT
Schematic and wiring	<code>hardware/</code>	CERN-OHL-P-2.0
Documentation	<code>docs/</code>	CC BY 4.0
Photos and animations	<code>media/</code>	CC BY 4.0

B.2 Third-party software

The firmware was written entirely by the author; no third-party source code is contained in the repository. The following libraries are required to build it:

Table B.2: Third-party software used by the firmware.

Component	Purpose	License
MPU6050_light 1.2.1 [4]	reading the IMU	MIT
Arduino core for the ESP32 3.3.10 [5]	Arduino API, I ² C, PWM, ADC	LGPL-2.1
BluetoothSerial (part of the core)	Bluetooth command interface	Apache-2.0
Preferences (part of the core)	storage in flash	Apache-2.0
ESP-IDF (basis of the core)	system software, Bluetooth stack	Apache-2.0

MIT and Apache-2.0 are permissive and compatible with the MIT license of the firmware. The LGPL-2.1 of the ESP32 core applies to compiled binaries that contain the core; since the complete source code of the firmware is published, recipients of a binary can rebuild it with a modified core. Further details are given in the file `NOTICE.md`.